

Методическое пособие по созданию FPS-шутера в Unity

Автор: Максаков Михаил

Дисциплина: Разработка игр на Unity (C#)

Введение

Данное методическое пособие предназначено для студентов, начинающих изучение разработки игр в Unity. В ходе выполнения практической работы вы создадите простую систему стрельбы от первого лица с активными целями.

Что вы изучите:

- Основы работы с GameObject и компонентами
- Написание скриптов на C# в Unity
- Использование Raycast (луча) для определения попаданий
- Создание Coroutines (сопрограмм) для анимации
- Управление курсором мыши
- Взаимодействие между скриптами

Теоретическая часть

1. Что такое Raycast?

Raycast (бросание луча) — это метод, при котором из точки в заданном направлении виртуально выпускается луч. Если луч пересекается с объектом, имеющим коллайдер, мы получаем информацию об этом объекте.

Где применяется:

- Определение, во что попала пуля
- Определение, на какой объект смотрит игрок
- Проверка видимости объектов
- Реализация прицеливания

2. Что такое Coroutine (сопрограмма)?

Сопрограмма — это метод, который может приостанавливать своё выполнение и возобновлять его через указанное время. Используется для создания задержек и анимаций без блокировки игры.

3. Компоненты GameObject

Каждый игровой объект состоит из компонентов:

- Transform — положение, поворот, масштаб
- Collider — отвечает за столкновения
- Mesh Renderer — отображает модель
- Custom Script — ваш собственный код

Практическая часть

Этап 1. Создание сцены

1.1 Создание игрока и камеры

1. Создайте пустой объект: `GameObject → Create Empty`
2. Назовите его `Player`
3. Установите позицию: `X=0, Y=0, Z=0`
4. Найдите в сцене `Main Camera`
5. Перетащите камеру внутрь объекта `Player`
6. Установите позицию камеры внутри Player: `X=0, Y=0.5, Z=0`

1.2 Создание пола

1. `GameObject → 3D Object → Plane`
2. Назовите `Floor`
3. Позиция: `X=0, Y=-0.5, Z=0`
4. Масштаб: `X=5, Y=1, Z=5`

1.3 Создание стен

Стена 1 (задняя):

- `GameObject → 3D Object → Cube`
- Позиция: `X=0, Y=1, Z=-8`
- Масштаб: `X=10, Y=2, Z=0.5`

Стена 2 (левая):

- Позиция: `X=-5, Y=1, Z=0`
- Масштаб: `X=0.5, Y=2, Z=16`

Стена 3 (правая):

- Позиция: `X=5, Y=1, Z=0`
- Масштаб: `X=0.5, Y=2, Z=16`

Стена 4 (передняя):

- Позиция: `X=0, Y=1, Z=8`
- Масштаб: `X=10, Y=2, Z=0.5`

1.4 Добавление света

`GameObject → Light → Directional Light`

- Поворот: `X=50, Y=-30, Z=0`

Этап 2. Создание скрипта MouseLook

Название скрипта: `MouseLook.cs`

Полный код:

```
``csharp
using UnityEngine;

public class MouseLook : MonoBehaviour
{
    [Header("Настройки чувствительности")]
    public float mouseSensitivity = 100f;

    [Header("Ограничение вертикального взгляда")]
    public float minVerticalAngle = -90f;
    public float maxVerticalAngle = 90f;

    [Header("Ссылки на объекты")]
    public Transform playerBody;

    private float xRotation = 0f;
    private float yRotation = 0f;

    void Start()
    {
        Cursor.lockState = CursorLockMode.Locked;
        Cursor.visible = false;
    }

    void Update()
    {
        float mouseX = Input.GetAxis("Mouse X") * mouseSensitivity *
Time.deltaTime;
```

```

        float mouseY = Input.GetAxis("Mouse Y") * mouseSensitivity *
Time.deltaTime;

        xRotation -= mouseY;
        xRotation = Mathf.Clamp(xRotation, minVerticalAngle,
maxVerticalAngle);
        yRotation += mouseX;

        transform.localRotation = Quaternion.Euler(xRotation, 0f, 0f);

        if (playerBody != null)
        {
            playerBody.rotation = Quaternion.Euler(0f, yRotation, 0f);
        }

        if (Input.GetKeyDown(KeyCode.Escape))
        {
            Cursor.lockState = CursorLockMode.None;
            Cursor.visible = true;
        }
        else if (Input.GetMouseButtonDown(0) && Cursor.lockState ==
CursorLockMode.None)
        {
            Cursor.lockState = CursorLockMode.Locked;
            Cursor.visible = false;
        }
    }
}

```

Что делает код:

- Блокирует курсор в центре экрана
- Поворачивает камеру при движении мыши
- Ограничивает поворот вверх и вниз (нельзя перевернуться)
- При нажатии Esc разблокирует курсор

Настройка в Unity:

1. Добавьте скрипт на камеру (Main Camera)
2. В поле 'Player Body' перетащите объект Player

Этап 3. Создание скрипта стрельбы RayShooter

Название скрипта: 'RayShooter.cs'

Полный код:

```
``csharp
using UnityEngine;
using System.Collections;

public class RayShooter : MonoBehaviour
{
    private Camera _camera;

    void Start()
    {
        _camera = GetComponent<Camera>();
    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            Shoot();
        }
    }

    private void Shoot()
    {
        Vector3 center = new Vector3(_camera.pixelWidth / 2,
        _camera.pixelHeight / 2, 0);
        Ray ray = _camera.ScreenPointToRay(center);
        RaycastHit hit;

        if (Physics.Raycast(ray, out hit))
        {
            GameObject hitObject = hit.transform.gameObject;
            ReactiveTarget target = hitObject.GetComponent<ReactiveTarget>();

            if (target != null)
            {
                target.ReactToHit();
            }
            else
            {
                StartCoroutine(SphereIndicator(hit.point));
            }
        }
    }
}
```

```

private IEnumerator SphereIndicator(Vector3 pos)
{
    GameObject sphere =
GameObject.CreatePrimitive(PrimitiveType.Sphere);
    sphere.transform.position = pos;
    yield return new WaitForSeconds(1);
    Destroy(sphere);
}

void OnGUI()
{
    int size = 12;
    float posX = _camera.pixelWidth / 2 - size / 4;
    float posY = _camera.pixelHeight / 2 - size / 2;
    GUI.Label(new Rect(posX, posY, size, size), "+");
}
}

```

Что делает код:

- При нажатии ЛКМ пускает луч из центра экрана
- Если луч попал в объект с компонентом `ReactiveTarget` → вызывает его реакцию
- Если попал в другой объект → показывает сферу на 1 секунду
- Рисует прицел (+) в центре экрана

Настройка в Unity:

- Добавьте скрипт на камеру (Main Camera)

Этап 4. Создание активной цели (врага)

4.1 Создание объекта Enemy

1. `GameObject → 3D Object → Cube`
2. Назовите `Enemy`
3. Позиция: `X=0, Y=1, Z=5`
4. Масштаб: `X=1, Y=2, Z=1`
5. Создайте красный материал и назначьте врагу

4.2 Создание скрипта ReactiveTarget

Название скрипта: `ReactiveTarget.cs`

Полный код:

```
```csharp
using UnityEngine;
using System.Collections;

public class ReactiveTarget : MonoBehaviour
{
 public void ReactToHit()
 {
 StartCoroutine(Die());
 }

 private IEnumerator Die()
 {
 this.transform.Rotate(-75, 0, 0);
 yield return new WaitForSeconds(1.5f);
 Destroy(this.gameObject);
 }
}
```
```

Что делает код:

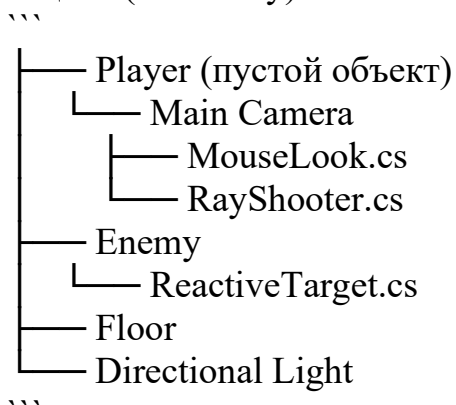
- При попадании запускает сопрограмму Die()
- Поворачивает врага на -75 градусов (он падает)
- Ждёт 1.5 секунды
- Уничтожает объект врага

Настройка в Unity:

- Добавьте скрипт на объект Enemy

Итоговая структура проекта

Сцена (Hierarchy):



Скрипты и их расположение:

Скрипт | На каком объекте

MouseLook.cs | Main Camera |

RayShooter.cs | Main Camera |

ReactiveTarget.cs | Enemy |

Проверка работоспособности

Пошаговая проверка:

1. Запустите игру (нажмите Play)
2. Проверьте курсор:
 - Указатель мыши должен исчезнуть
 - Движение мыши должно поворачивать камеру
3. Проверьте прицел:
 - В центре экрана должен быть символ "+"
4. Проверьте стрельбу по стене:
 - Нажмите ЛКМ, глядя на стену
 - В точке попадания должна появиться серая сфера
 - Сфера исчезнет через 1 секунду
5. Проверьте стрельбу по врагу:
 - Наведите прицел на красный куб
 - Нажмите ЛКМ
 - Враг должен наклониться и через 1.5 секунды исчезнуть
6. Проверьте разблокировку курсора:
 - Нажмите клавишу Esc
 - Курсор должен появиться
 - Кликните ЛКМ на сцене → курсор снова заблокируется

Контрольные вопросы

1. Что такое Raycast и для чего он используется?
2. Какая разница между `Update()` и `Start()`?
3. Зачем нужна сопрограмма (Coroutine)?
4. Что делает метод `GetComponent<>()`?
5. Как разблокировать курсор мыши во время игры?
6. Почему камера вложена в объект Player?
7. Что такое `yield return new WaitForSeconds()`?